



partial_sort_n, nth_element_n

Document number:

[P3735R0](#)

Date:

2025-06-28

Audience:

SG9

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3735/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/n-algorithms.cow



ABSTRACT: The `partial_sort`, and `nth_element` algorithms require the user to "manually offset" iterators, which is neither ergonomic nor safe. I propose `_n` variants of these algorithms in the style of `shift_left`.

§ Contents

- 1 **Introduction**
 - 1.1 Middle iterators are unsafe
 - 1.2 Middle iterators are annoying
- 2 **Scope**
- 3 **Design**
- 4 **Impact on existing code**
- 5 **Wording**
- 6 **References**

§ 1. Introduction

Algorithms such as `std::ranges::partial_sort` accept a "middle iterator" that must be in the range of iterators `[begin(), end())`. For example, `ranges::partial_sort` is specified in [\[partial.sort\]](#) as follows:

```
template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::partial_sort(I first, I middle, S last, Comp comp = {}, Proj proj =
```

```
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::partial_sort(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj
```

There are problems with this interface, described below.

§ 1.1. Middle iterators are unsafe

A common use case for `partial_sort` or `nth_element` is to sort the N lowest elements or to obtain the N^{th} lowest element, where N is an integer that is constant, obtained from a config file, or otherwise independent of the range. This invites a common bug, which I have witnessed multiple times:



BUG: The following code has undefined behavior.

```
const int n = 10;
// ...

// usually obtained from user input, configs, etc.:
int scores[] { 5, 2, 8, 0, 3 };
// now, to obtain the 10 greatest scores:
std::partial_sort(scores,
                  scores + n, // 🚫 undefined behavior
                  std::end(scores),
                  std::greater<int>{});
// same bug with std::ranges:
std::ranges::partial_sort(scores, scores + n, std::ranges::greater{});
```

The underlying problem is that to use the algorithm, we need to offset the start iterator, and this can fall outside of the range of iterators `[begin(), end())`.

A possible safe alternative which would have prevented the bug above is

```
std::ranges::partial_sort(scores,
                          std::ranges::next(scores, n, std::ranges::end(scores)),
                          std::ranges::greater{});
```

However, this is neither ergonomic nor does it address the cause of the bug. The bug was caused by the user not remembering that `scores + n` can be undefined, and by the interface encouraging them to write this expression. Why should we expect the user to know of and consistently use this workaround then?

Users generally prefer to use the "path of least resistance", and we should make that path safe:

```
// OK, even if n is greater than the size of the range
std::ranges::partial_sort_n(scores, n, std::ranges::greater{});
```

§ 1.2. Middle iterators are annoying

Since we need to obtain the middle iterator from the range but also provide the range to `partial_sort_n`, we may have to repeat ourselves.



EXAMPLE: Say we want to obtain the N lowest scores similar to the bug above, but the scores are obtained from a function instead:

```
namespace rgs = std::ranges;
some_range& long_function_to_get_scores_range();

// without a variable:
rgs::partial_sort(long_function_to_get_scores_range(),
    rgs::next(rgs::begin(long_function_to_get_scores_range()), n, // 🐛
    rgs::end(long_function_to_get_scores_range())),
    rgs::greater{});

// with a variable:
some_range& r = long_function_to_get_scores_range();
rgs::partial_sort(r, rgs::next(rgs::begin(r), n, rgs::end(r)), rgs::greater{});
```

Either options is somewhat annoying if it could theoretically be written as:

```
rgs::partial_sort_n(long_function_to_get_scores_range(), n, rgs::greater{});
```

One key benefit of `std::ranges` is that we rarely have to work with iterators directly anymore. Requiring the user to provide middle iterators somewhat defeats this design strategy.

§ 2. Scope

To address the problems above, I propose the following additional algorithms:

- `partial_sort_n`
- `nth_element_n`

These should exist both for the `std::ranges::` and the `std::` algorithms, as they provide a "safety/ergonomics hotfix" for anyone who has not migrated to these "new" algorithms yet, and is perhaps unable to because their iterators would not satisfy the `std::ranges` constraints. 

While there are more algorithms that accept middle iterators, they do not exclusively operate on random access iterators, which results in different design concerns. There is also no proposed overload for `partial_sort_copy` because its result can already be specified as a range or iterator pair, so the concerns in §1.1. [Middle iterators are unsafe](#) do not apply.



DECISION: Depending on SG9 feedback, the scope of this paper could be widened to provide alternatives to middle iterators across `<algorithm>` as a whole.

§ 3. Design

While the names follow the scheme of `copy_n`, the design is essentially copied from `shift_left`, which is the `_n` counterpart to `rotate`. `copy_n` takes an iterator and the size of the range, without an end iterator, and thus without any safety benefits.

The proposed functions would take either an iterator pair and a size, or a range and a size, just like `shift_left` and `shift_right`, and with similar interface. See [\[alg.shift\]](#).

§ 4. Impact on existing code

No existing code should be affected because the proposed functions are all new. Existing overload sets are not modified.

§ 5. Wording

The proposed changes are relative to [\[N5008\]](#).

In [\[version.syn\]](#), add two feature-test macros as follows:



```
#define __cpp_lib_ranges_partial_sort_n 20XXXXL // also in <algorithm>
#define __cpp_lib_ranges_nth_element_n 20XXXXL // also in <algorithm>
```

In [\[algorithm.syn\]](#), change the synopsis of `<algorithm>` as follows:



```
namespace std {
    [...]

    template<class RandomAccessIterator>
        constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator last,
                                     RandomAccessIterator last);

    template<class RandomAccessIterator, class Compare>
```



```

constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec, // freestanding
                  RandomAccessIterator first, RandomAccessIterator middle,
                  RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec, // freestanding
                  RandomAccessIterator first, RandomAccessIterator middle,
                  RandomAccessIterator last, Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
              class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
        partial_sort(I first, I middle, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
        partial_sort(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
void partial_sort_n(RandomAccessIterator first, RandomAccessIterator last,
                    typename iterator_traits<RandomAccessIterator>::difference_type n,
                    Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort_n(ExecutionPolicy&& exec, // freestanding
                    RandomAccessIterator first, RandomAccessIterator last,
                    typename iterator_traits<RandomAccessIterator>::difference_type n,
                    Compare comp);
template<class RandomAccessIterator, class Compare>
void partial_sort_n(RandomAccessIterator first, RandomAccessIterator last,
                    typename iterator_traits<RandomAccessIterator>::difference_type n,
                    Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort_n(ExecutionPolicy&& exec, // freestanding
                    RandomAccessIterator first, RandomAccessIterator last,
                    typename iterator_traits<RandomAccessIterator>::difference_type n,
                    Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
              class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
        partial_sort_n(I first, iter_difference_t<I> n, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
        partial_sort_n(R&& r, ranges::range_difference_t<R> n, Comp comp = {}, Proj proj = {});
}

[...]
```



```
// [alg.nth.element] Nth element
template<class RandomAccessIterator>
    constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                               RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
    constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                               RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
    void nth_element(ExecutionPolicy&& exec,                               // freestanding
                     RandomAccessIterator first, RandomAccessIterator nth,
                     RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element(ExecutionPolicy&& exec,                               // freestanding
                     RandomAccessIterator first, RandomAccessIterator nth,
                     RandomAccessIterator last, Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = random_access_iterator_traits<I>::difference_type, class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            nth_element(I first, I nth, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            nth_element(R&& r, iterator_t<R> nth, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
    void nth_element_n(RandomAccessIterator first, RandomAccessIterator last,
                       typename iterator_traits<RandomAccessIterator>::difference_type n);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element_n(ExecutionPolicy&& exec, RandomAccessIterator first,
                       RandomAccessIterator last,
                       typename iterator_traits<RandomAccessIterator>::difference_type n,
                       Compare comp);
template<class RandomAccessIterator, class Compare>
    void nth_element_n(RandomAccessIterator first, RandomAccessIterator last,
                       typename iterator_traits<RandomAccessIterator>::difference_type n,
                       Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element_n(ExecutionPolicy&& exec, RandomAccessIterator first,
                       RandomAccessIterator last,
                       typename iterator_traits<RandomAccessIterator>::difference_type n,
                       Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = random_access_iterator_traits<I>::difference_type, class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            nth_element_n(I first, S last, iter_difference_t<I> n, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            nth_element_n(R&& r, S last, iter_difference_t<R> n, Comp comp = {}, Proj proj = {});
}
```

```

    constexpr borrowed_iterator_t<R>
        nth_element_n(R&& r, ranges::range_difference_t<R> n, Comp comp =
    }

    [...]
}

```

In `[partial.sort]`, append the following items at the end of the subclause:



```

template<class RandomAccessIterator>
    void partial_sort_n(RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void partial_sort_n(ExecutionPolicy&& exec,
        RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class RandomAccessIterator, class Compare>
    void partial_sort_n(RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void partial_sort_n(ExecutionPolicy&& exec,
        RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);

```

Effects: Equivalent to calling `partial_sort` with all parameters excluding `n` in the same order, except that `first + min<Size>(n, last - first)` is provided as an argument for the middle parameter in `partial_sort`.

```

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
        ranges::partial_sort_n(I first, iter_difference_t<I> n, S last, Comp comp,
            Proj proj = {});

```

Effects: Equivalent to:

```

    return ranges::partial_sort(first, ranges::next(first, n, last), last, comp, proj);

```

```

template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
        ranges::partial_sort_n(R&& r, ranges::range_difference_t<R> n, Comp comp,
            Proj proj = {});

```

Effects: Equivalent to:

```

    return ranges::partial_sort_n(ranges::begin(r), n, ranges::end(r), comp, proj);

```


In [alg.nth.element], append the following items at the end of the subclause:



```
template<class RandomAccessIterator>
    void nth_element_n(RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element_n(ExecutionPolicy&& exec, RandomAccessIterator first,
        RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class RandomAccessIterator, class Compare>
    void nth_element_n(RandomAccessIterator first, RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element_n(ExecutionPolicy&& exec, RandomAccessIterator first,
        RandomAccessIterator last,
        typename iterator_traits<RandomAccessIterator>::difference_type n,
        Compare comp);
```

Effects: Equivalent to calling `nth_element` with all parameters excluding `n` in the same order, except that `first + min<Size>(n, last - first)` is provided as an argument for the `nth` parameter in `nth_element`.

```
template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
    class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
        ranges::nth_element_n(I first, S last, iter_difference_t<I> n, Comp comp,
        Proj proj = {});
```

Effects: Equivalent to:

```
return ranges::nth_element(first, ranges::next(first, n, last), last, comp, proj);
```

```
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
        ranges::nth_element_n(R&& r, ranges::range_difference_t<R> n, Comp comp,
        Proj proj = {});
```

Effects: Equivalent to:

```
return ranges::nth_element_n(ranges::begin(r), n, ranges::end(r), comp, proj);
```

§ 6. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>